Estudio comparativo del rendimiento de un árbol de decisión C4.5 propio y del modelo de ensamble Random Forest en problemas de clasificación

Laura Cornaló Bassan[1 [00000]], Matías Fernandez[1 [24282]], Mariano Insaurralde[1 [24684]], Philippe Maurel[1 [25613]], y Franco Sanchez[1 [27310]]

Universidad Tecnológica Nacional, Facultad Regional de Resistencia
Ingeniería en Sistemas de Información
{correo1,correo2,correo3,correo4,correo5}@gmail.com

Resumen hola soy un abstract

Palabras clave: Árbol de Decisión · C4.5 · Random Forest

1 Introducción

Un clasificador automático asigna una clase a cada caso a partir de sus características. Los árboles de decisión producen un modelo legible, en el que cada camino de la raíz a una hoja equivale a una regla [2]. Random Forest combina muchos árboles para reducir la dependencia de una sola estructura, a cambio de más cómputo y de una decisión que ya no se sigue en un único camino [1]. C4.5 elige cada pregunta con un criterio orientado a separar las clases y admite atributos numéricos y categóricos [2]. Ninguno de los dos enfoques domina al otro en general: la elección depende del problema, de la métrica que se priorice y del costo que se acepte pagar.

Este trabajo compara una implementación propia de C4.5 con el Random Forest de scikit-learn [3] sobre el conjunto de datos [DATASET PENDIENTE]. La comparación usa las mismas particiones, las mismas métricas y las mismas condiciones de evaluación. Con esquemas de evaluación distintos, una diferencia de rendimiento puede reflejar la forma de medir y no el algoritmo. El objetivo es determinar en qué condiciones conviene cada uno, atendiendo a la tasa de acierto (proporción de predicciones correctas), la estabilidad de las predicciones ante cambios de semilla, el tiempo de cómputo y la interpretabilidad.

Las contribuciones son una implementación propia de C4.5 con decisiones explícitas sobre la construcción, el criterio de división y el control del crecimiento; un protocolo común de evaluación; y un análisis comparativo de esos cuatro aspectos.

El documento describe primero los fundamentos de ambos modelos, luego la implementación, después los experimentos y cierra con las conclusiones.

2 Fundamentos teóricos

2.1 Árboles de decisión y C4.5

C4.5 construye un árbol de decisión de forma recursiva [2]. En cada nodo elige la pregunta sobre un atributo que mejor separa las clases y repite el procedimiento en los subconjuntos resultantes. La entropía mide la impureza de un nodo a partir de la distribución de sus clases. Si D es el conjunto de casos, C el conjunto de clases y p_c la proporción de casos de la clase c, medida por masa cuando los pesos no son unitarios, la entropía se define en la ecuación (1):

$$H(D) = -\sum_{c \in C} p_c \log_2 p_c. \tag{1}$$

La entropía vale cero cuando todos los casos pertenecen a una misma clase y alcanza su máximo cuando las clases están equilibradas.

Una pregunta sobre un atributo A divide D en v subconjuntos D_j, con masas W_j, y W es la masa de D. La ganancia de información es la reducción de entropía que produce esa división, ecuación (2), donde la suma ponderada representa la impureza que queda en las ramas. Cuanto mayor es la ganancia, más puras quedan las ramas, y una ganancia nula indica que la partición no reduce la impureza.

$$\mathrm{Gain}(D, A) = H(D) - \sum_{j=1}^{v} \frac{W_j}{W} H(D_j). \tag{2}$$

La ganancia favorece a los atributos con muchos valores distintos, porque fragmentar el conjunto reduce la impureza casi por construcción. C4.5 atenúa ese efecto con la información de la partición, ecuación (3), que mide cuánto fragmenta el atributo. La tasa de ganancia, ecuación (4), es el cociente entre ambas magnitudes; un atributo que fragmenta demasiado queda penalizado. Si la partición no produce al menos dos ramas conocidas de masa positiva, el atributo no se considera candidato.

$$\mathrm{SplitInfo}(D, A) = -\sum_{j=1}^{v} \frac{W_j}{W} \log_2 \frac{W_j}{W}, \tag{3}$$

$$\mathrm{GainRatio}(D, A) = \frac{\mathrm{Gain}(D, A)}{\mathrm{SplitInfo}(D, A)}. \tag{4}$$

Los atributos numéricos requieren un tratamiento distinto. En cada nodo se ordenan los valores observados, operación de costo $O(|D| \log |D|)$, y se examinan los $r - 1$ cortes entre valores consecutivos; cada corte separa los casos con $A \leq t$ de los que tienen $A > t$. Se conserva el corte con la mayor tasa de ganancia. El umbral es un valor observado y se elige en cada nodo, sin discretización global. Un atributo numérico puede reutilizarse en niveles inferiores, mientras que un atributo categórico se descarta después de usarse [2].

Con valores faltantes, sea W_K la masa de los casos con valor conocido para el atributo y W_U la de los que no lo tienen. La ganancia se calcula solo sobre

el subconjunto D_K de casos conocidos y se escala por la fracción W_K/W, ecuación (5). La información de la partición agrega la masa faltante W_U como una rama de cálculo, aunque no origine una rama del árbol. La tasa de ganancia ajustada es el cociente entre la ganancia ajustada y esa información de la partición. En el entrenamiento, un caso faltante se envía a todas las ramas y su peso se reparte según $q_j = W_j/W_K$, ecuación (6); en la predicción, las probabilidades de las ramas se combinan con esas mismas proporciones [2].

$$\text{Gain}^*(D, A) = \frac{W_K}{W}\left[H(D_K) - \sum_j \frac{W_j}{W_K} H(D_j)\right], \tag{5}$$

$$q_j = \frac{W_j}{W_K}. \tag{6}$$

Este mecanismo es propio de C4.5.

Un árbol que crece sin restricciones puede ajustarse a rasgos particulares del conjunto de entrenamiento y perder capacidad sobre casos nuevos. La prepoda detiene el crecimiento durante la construcción; la pospoda elimina subárboles después de construir el árbol completo. El C4.5 original emplea pospoda pesimista, mientras que este trabajo aplica solo prepoda y no implementa poda de reglas [2]. La construcción es codiciosa: cada división se elige con la información disponible en el nodo y no se revisa después, de modo que la prepoda puede detener el crecimiento antes de lo conveniente.

2.2 Random Forest

Random Forest combina B árboles de decisión construidos con perturbaciones aleatorias [1]. Cada árbol se entrena con una muestra del mismo tamaño que el conjunto original, obtenida con reemplazo, y en cada división evalúa solo un subconjunto aleatorio de los atributos disponibles. Los árboles crecen sin poda y la predicción combina sus respuestas por voto mayoritario o por promedio de probabilidades. Esa doble fuente de aleatoriedad reduce la dependencia de una sola estructura y la correlación entre los árboles.

Breiman muestra que el error de generalización depende de la fuerza de los árboles y de su correlación: árboles fuertes y poco correlacionados tienden a reducir el error [1]. La fuerza resume la separación media entre la clase correcta y las alternativas, y la correlación mide cuánto se parecen las respuestas de dos árboles. El análisis de este trabajo es empírico y no usa esa caracterización para calcular el error. En este trabajo se usa la implementación de scikit-learn (...); su configuración se detalla en la sección 5.

3 Breve descripción de los algoritmos desarrollados

3.1 C4.5

El Algoritmo 1 resume la construcción del árbol. El procedimiento recibe el conjunto de casos, el conjunto de atributos disponibles $\mathcal{A}$, la profundidad d, los

cuatro controles de crecimiento y devuelve un nodo. La masa de un nodo es la suma de los pesos de sus casos, de modo que los casos repartidos por valores faltantes no cuentan como filas completas. Las paradas estructurales se evalúan antes de elegir la división; la comparación con el umbral de ganancia y con la tasa mínima se aplica después, y es estricta: si el valor es menor, el crecimiento se detiene; la igualdad permite dividir.

Algoritmo 1 Construcción recursiva de un árbol C4.5

Calcular la masa por clase y elegir la clase mayoritaria.
si clase única, $\mathcal{A}$ vacío, profundidad máxima o masa mínima **entonces**
    devolver hoja con la clase mayoritaria.
fin si
para cada atributo $a \in \mathcal{A}$ **hacer**
    Evaluar sus divisiones y conservar la mayor tasa de ganancia ajustada.
fin para
si no hay candidato o no se alcanzan los mínimos **entonces**
    devolver hoja con la clase mayoritaria.
fin si
Crear el nodo con el atributo, el umbral o las ramas, y las proporciones.
para cada rama **hacer**
    Repartir los faltantes y construir el hijo con $d + 1$.
fin para
devolver el nodo.

Los empates se resuelven por el orden de los atributos y, entre cortes numéricos del mismo atributo, por el menor umbral. Un atributo categórico se elimina del conjunto disponible después de usarse; uno numérico se conserva, porque los subconjuntos resultantes admiten nuevos cortes. En la predicción, un valor conocido sigue una sola rama; un valor faltante combina las distribuciones de todas las ramas con las proporciones aprendidas, y una categoría no vista usa las frecuencias locales del nodo alcanzado. La clase final es la de mayor probabilidad y los empates se resuelven con la primera clase observada en el entrenamiento.

3.2 Random Forest

La implementación de Random Forest usa scikit-learn ...

4 Descripción de la solución implementada

4.1 Arquitectura y protocolo de evaluación

La solución se organiza en tres capas: preparación de los datos, modelos y evaluación. La preparación carga el conjunto de datos, deduce el tipo de cada atributo y construye una partición estratificada con el 80 % de los casos para entrenamiento y el 20 % para prueba, con una semilla fija de 42. Ambos modelos

usan las mismas particiones y las mismas métricas, y ninguno accede al conjunto de prueba durante el entrenamiento. C4.5 admite valores faltantes y categorías nativas. El tratamiento de valores faltantes en Random Forest se definirá junto con la implementación de ese modelo. Random Forest transforma las categorías en columnas binarias dentro de su flujo.

La codificación de Random Forest se ajusta solo con el conjunto de entrenamiento y una categoría no vista se representa con ceros; C4.5 conserva una rama por categoría. Esas diferencias de preprocesamiento son una posible fuente de sesgo en la comparación y se retoman al interpretar los resultados. Ambos modelos exponen el mismo contrato de predicción: reciben una tabla de datos, devuelven una columna con la clase predicha y conservan el índice y el orden de las filas sin modificar la entrada. La validación exige las mismas columnas y tipos del entrenamiento, y los desempates del árbol son deterministas. Las probabilidades del árbol son frecuencias ponderadas y no están calibradas.

4.2 Decisiones de diseño y problemas encontrados

Los principales problemas fueron el tratamiento de atributos continuos, los valores faltantes, el control del crecimiento y la comparabilidad del preprocesamiento. La Tabla 1 resume los tratamientos adoptados o previstos. Las pruebas de cortes, conservación de masa y repetición de particiones respaldan los tratamientos de mayor riesgo. El crecimiento se controla con cuatro parámetros independientes: el umbral de ganancia, la profundidad máxima, la masa mínima y la tasa de ganancia mínima; no se implementa pospoda.

Tabla 1. Problemas técnicos y tratamientos adoptados o previstos.

Problema	Tratamiento
Atributos continuos	Cortes binarios en valores observados; sin grilla global
Valores faltantes	C4.5 los reparte con peso proporcional
Crecimiento	Prepoda con cuatro controles; sin pospoda
Preprocesamiento	Categorías nativas en C4.5; codificación binaria en Random Forest

5 Simulaciones y resultados

6 Conclusiones

Referencias

1. Breiman, L.: Random forests. Machine Learning **45**(1), 5–32 (2001). https://doi.org/10.1023/A:1010933404324

2. Liu, B.: Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data. Data-Centric Systems and Applications, Springer, Berlin, Heidelberg, 2 edn. (2011). https://doi.org/10.1007/978-3-642-19460-3
3. scikit-learn developers: Random forests. `https://scikit-learn.org/stable/modules/ensemble.html#forest`, accessed 2026-10-15